Технический анализ ветки `main` — USV Obstacle Detection

Репозиторий: DmitriyZhariy/USV-Obstacle-Detection
Авторы: Dmitry Zharyj, Evgeny Zagidulin
Версия: 0.1.0 | **Лицензия:** MIT | **Статус:** research-preview

---

1. Общее описание ветки `main`

Назначение и контекст

Ветка `main` служит базовым шаблоном (scaffolding) проекта, реализованного в рамках магистерской диссертации на тему *«Разработка программного модуля обнаружения и пространственной локализации надводных препятствий для беспилотного надводного судна (USV) на основе интеллектуальной мультисенсорной обработки данных»*. Главная цель системы — построить исследовательский конвейер (research-grade pipeline), который обрабатывает сырые данные со стереокамер и сенсоров (IMU, GPS) и генерирует локальную карту препятствий для навигации.

Ветка не содержит никакой реальной логики обнаружения препятствий. Вся прикладная разработка сосредоточена в производных ветках `feat/synchronization` и `feat/segmentation-dataset`. Роль `main` — зафиксировать единую среду окружения, структуру директорий и инструменты контроля качества кода, от которой отпочковываются все тематические ветки.

Этап разработки

На момент последнего коммита (01.12.2025) ветка `main` находится на самом начальном этапе — инициализации проекта. Об этом свидетельствует `main.py`, содержащий единственную строку `print("Hello from diploma-project!")`, пустой файл `configs/config.yaml` и директория `models/`, закреплённая

лишь заглушкой .gitkeep. Бейдж status: research-preview в README подтверждает, что проект находится в стадии исследовательского прототипа.

---

2. Архитектура проекта

Структура директорий и файлов

Проект следует модифицированной структуре Cookiecutter Data Science:

```
USV-Obstacle-Detection/
├── .gitignore
├── .pre-commit-config.yaml     # Конфигурация pre-commit хуков
├── .python-version             # Фиксация версии Python (3.11)
├── CITATION.cff                # Метаданные для академического цитирован
├── LICENSE                     # MIT License
├── README.md
├── main.py                     # Точка входа (заглушка)
├── pyproject.toml              # Зависимости и метаданные пакета
├── setup.py                    # Пустой файл (legacy)
├── uv.lock                     # Зафиксированные версии зависимостей
├── configs/
│   └── config.yaml             # Пустой конфиг (Hydra, заготовка)
├── models/
│   └── .gitkeep                # Директория для весов модели (пустая)
└── src/
    └── usv/
        └── __init__.py         # Пустой инициализатор пакета
```

Директории data/, notebooks/ и reports/ фигурируют в README как часть проектируемой архитектуры, но в main физически отсутствуют (входят в .gitignore или ещё не созданы).

Разделение на модули и их ответственность

Planned-архитектура, описанная в README, предполагает следующее разделение:

Модуль	Путь	Ответственность
data	src/usv/data/	Загрузка и парсинг сырых видео/сенсорных данных
preprocessing	src/usv/preprocessing/	Калибровка, синхронизация, ректификация
perception	src/usv/perception/	Нейросети: сегментация и детекция
mapping	src/usv/mapping/	3D-реконструкция, генерация карты препятствий

В самой ветке main ни один из этих подпакетов физически не заполнен — существует лишь корневой src/usv/__init__.py. Реализация появляется в производных ветках.

Поток обработки данных (проектируемый)

Описанный в README конвейер предполагает следующий поток от сенсоров до карты препятствий:

```
┌──────────────────────────────────────────────────────────────────────────────┐
│  ИСТОЧНИКИ ДАННЫХ                                                            │
│  [Стереокамера (L/R)] ──┐                                                    │
│  [Телефон (центр)]  ────┼──→ [data/raw/]                                     │
│  [IMU / GPS]        ────┘                                                    │
└──────────────────────────────────────────────────────────────────────────────┘
                  │
                  ▼
┌──────────────────────────────────────────────────────────────────────────────┐
│  PREPROCESSING (src/usv/preprocessing/)                                      │
│  1. Синхронизация потоков (аудио-корреляция)                                 │
│  2. Калибровка камер (intrinsics/extrinsics)                                 │
│  3. Ректификация стереопары                                                  │
```

```
│  ↓ data/processed/                                  │
└──────────────────────────────────────────────────────────────────────────────
          │
          ▼
┌──────────────────────────────────────────────────────────────────────────────
│  PERCEPTION (src/usv/perception/)                          │
│  4. Сегментация/Детекция препятствий (YOLOv8-seg)                 │
│  5. Вычисление диспаритета (Stereo Matching)                    │
│  ↓ Bounding boxes, маски, depth maps                          │
└──────────────────────────────────────────────────────────────────────────────
          │
          ▼
┌──────────────────────────────────────────────────────────────────────────────
│  MAPPING (src/usv/mapping/)                                │
│  6. Triangulation → 3D-координаты препятствий                    │
│  7. Генерация локальной карты навигации                        │
│  ↓ Карта / управляющие сигналы                                │
└──────────────────────────────────────────────────────────────────────────────
```

Шаги 1–5 частично реализованы в производных ветках; шаги 6–7 (модуль `mapping`) остаются на стадии проектирования.

---

3. Используемые зависимости

Основные производственные зависимости

Все зависимости объявлены в `pyproject.toml` и зафиксированы в `uv.lock`:

Библиотека	Версия	Роль в системе
torch	>=2.9.1	Базовый фреймворк для нейросетевых моделей (YOLOv8 backend)
torchvision	>=0.24.1	Трансформации изображений, предобученные модели CV
opencv-python	>=4.11.0.86	Захват видео, геометрические преобразования, SuperRes DNN
numpy	>=2.3.5	Матричные операции, обработка сигналов
scipy	>=1.16.3	Цифровая фильтрация (Butterworth), кросс-корреляция (fftconvolve)
pandas	>=2.3.3	Манифесты данных, таблицы временных меток
matplotlib	>=3.10.7	Визуализация результатов, графики синхронизации
hydra-core	>=1.3.2	Управление конфигурацией через YAML-файлы

Зависимости для разработки

Библиотека	Версия	Роль
pytest	>=9.0.1	Тестирование (заготовка)
ruff	>=0.14.7	Линтер и форматтер Python-кода
pre-commit	>=4.5.0	Git-хуки для контроля качества коммитов
ipykernel	>=7.1.0	Jupyter-ядро для исследовательских ноутбуков

Зависимости в производных ветках

В ветках feat/segmentation-dataset и feat/synchronization использованы дополнительные библиотеки, не объявленные в pyproject.toml ветки main:

- **ultralytics** — Python API для YOLOv8: загрузка предобученных весов, обучение, инференс
- **librosa** — загрузка и ресемплинг аудио из видеофайлов (16 кГц)
- **noisereduce** — спектральное шумоподавление (spectral gating) аудиодорожек
- **ffmpeg-python** — точное извлечение метаданных видеофайлов через ffprobe
- **tqdm** — прогресс-бар при обработке больших датасетов

Это расхождение между pyproject.toml и фактически используемыми зависимостями является техническим долгом: файл конфигурации пакета не синхронизирован с реальными зависимостями рабочих модулей.

Обоснование выбора ключевых библиотек

YOLOv8 (ultralytics) выбран как baseline-модель по совокупности характеристик: поддержка instance segmentation (флаг yolov8s-seg.pt), единый API для обучения и инференса, возможность дообучения на собственном датасете в формате YOLO-txt. Для задачи USV особенно важна поддержка retina_masks=True, дающая более качественные границы маски воды/препятствий. **SciPy** использован для цифровой фильтрации Баттерворта и быстрой кросс-корреляции через fftconvolve — стандартный научный инструментарий для обработки сигналов. **Hydra** выбран как система управления конфигурациями, популярная в ML-проектах: позволяет переопределять параметры из командной строки без изменения кода.

---

4. Конкретные реализации и анализ кода

> **Примечание:** в ветке `main` нет рабочего кода — вся реализация сосредоточена в производных ветках. Следующий раздел анализирует код, созданный на основе `main` и демонстрирующий заложенную архитектуру.

4.1 Подход к детекции препятствий

Проект использует **гибридный нейросетевой подход**:

- Первый этап — инстанс-сегментация водной поверхности и препятствий с помощью **YOLOv8-seg** (данные ветки feat/segmentation-dataset);
- Второй этап — **стереозрение**: глубина вычисляется из диспаратности стереорары (ещё не реализовано, обозначено в README);
- Дополнительный контекст обеспечивают **IMU и GPS** для ориентации судна.

Классические методы CV (Canny, Hough) не применяются — проект изначально ориентирован на deep learning как основу pipeline.

4.2 Модуль синхронизации (feat/synchronization)

`AudioAligner` — ключевой класс синхронизации разнородных видеопотоков:

```
class AudioAligner:
    def __init__(self, sample_rate: int = 16000):
        self.sr = sample_rate

    def clean_signal(self, data):
        # 1. Spectral Gating (noisereduce) — убирает фоновый шум двигателя
        data_clean = nr.reduce_noise(y=data, sr=self.sr, prop_decrease=0.7)
        # 2. Полосовой фильтр 2500–7500 Гц (isolates the sharp "clap" click)
        b, a = scipy.signal.butter(5, [low, high], btype='band')
        return scipy.signal.lfilter(b, a, data_clean)
```

```
def find_offset(self, ref_sig, tgt_sig):
    # Z-score нормализация → кросс-корреляция через FFT свертку
    correlation = scipy.signal.fftconvolve(ref_norm, tgt_norm[::-1], mode='fu
    lag_samples = lags[np.argmax(correlation)]
    return lag_samples / self.sr, correlation[np.argmax(correlation)]
```

Алгоритм включает два шага очистки сигнала: spectral gating (шумоподавление с коэффициентом prop_decrease=0.7) и полосовой фильтр Баттерворта 5-го порядка с полосой пропускания 2500–7500 Гц. Выбор этой полосы принципиален: он позволяет выделить резкий щелчок хлопка на фоне низкочастотного гула двигателя (ниже 300 Гц). Нормализация по Z-оценке устраняет влияние разных уровней громкости камер, а алгоритм fftconvolve с mode='full' обеспечивает O(N log N) вместо $O(N^2)$ прямой корреляции.

SyncPipeline использует телефон как *master reference*: находит аудиосмещение каждой dashcam-камеры относительно первого файла телефона и последовательно распространяет временные метки.

FullDatasetResync — улучшенная версия: синхронизирует каждый файл dashcam индивидуально против всего таймлайна телефона. При корреляции ниже порога 10.0 используется fallback-стыковка к концу предыдущего видео.

4.3 Подготовка датасета (feat/segmentation-dataset)

VideoManifestGenerator сканирует директории с сырыми видео, определяет тип камеры по именованию пути/файла и извлекает точную длительность через ffprobe. Логика извлечения временных меток учитывает специфику телефона Android: имя файла VID_YYYYMMDD_HHMMSS.mp4 кодирует время *завершения* записи, а не начала. Скрипт корректно вычисляет start_time = end_time - duration.

extract_frames.py извлекает кадры из видео с заданным интервалом и автоматически корректирует ориентацию телефонных видео:

```python
def extract_mixed_frames(input_dirs, output_dir, interval_sec=1):
    # Определение типа камеры по названию папки/файла
    if "phone" in filename_lower or "center" in filename_lower:
        prefix = "center"
    # Ротация для вертикальных видео с телефона
    if prefix == 'center':
        frame = cv2.rotate(frame, cv2.ROTATE_90_CLOCKWISE)
    # Именование с префиксом источника: left_00001.jpg, center_00002.jpg
    fname = f"{prefix}_{global_idx:05d}.jpg"
```

convert_mastr.py конвертирует датасет MaSTr1325 в формат YOLO-сегментации. Маппинг классов: пиксель со значением 0 → Obstacle (класс 1), значение 1 → Water (класс 0). Контуры извлекаются через cv2.findContours и аппроксимируются полигонами через cv2.approxPolyDP с эпсилоном 0.005 * perimeter, фильтруя шумовые контуры площадью менее 50 пикселей.

enhance_images_alg.py реализует дифференцированное улучшение изображений в зависимости от источника:

Тип камеры	Метод улучшения	Обоснование
Телефон (4K)	CLAHE (clip=2.0) + Bilateral Filter	Телефон уже HD, нужна лишь нормализация контраста
Dashcam (SD)	Gamma Correction (γ=1.3) + Bilateral Filter	Осветление недоэкспонированной воды/берега без засветки

CLAHE (Contrast Limited Adaptive Histogram Equalization) применяется в пространстве LAB: усиление контраста только по каналу яркости L, без влияния на цветовые каналы A и B.

upscale_images_ml.py повышает разрешение dashcam-кадров с помощью OpenCV DNN SuperRes: FSRCNN x3 (высокая скорость)

или EDSR x4 (высокое качество). Телефонные кадры пропускаются — их разрешение уже достаточно.

train_baseline.py запускает обучение YOLOv8s-seg на конвертированном датасете MaSTr1325:

```
model = YOLO('yolov8s-seg.pt')  # Small variant с сегментацией
results = model.train(
    data=dataset_yaml,
    project="reports/models",
    name="mastr_baseline",
    epochs=50,
    imgsz=640,
    batch=4,
    patience=10,  # Early stopping
    device='cpu'  # Временное решение
)
```

predict_video.py выполняет инференс обученной модели на видеофайле с поддержкой ротации и масштабирования для совместимости с телефонными видео. Параметр retina_masks=True активирует более точный алгоритм отрисовки масок. Результат — аннотированный MP4-файл с полигонными масками.

4.4 Обработка входного потока

Входными данными служат три видеопотока:

- data/raw/left_cam/ — левая dashcam (SD-разрешение);
- data/raw/right_cam/ — правая dashcam (SD-разрешение);
- data/raw/center_phone/ — центральный телефон (4K, вертикальная ориентация).

Телефон выполняет роль *master anchor* для временной синхронизации: его файлы содержат закодированные временные метки в имени, и именно они служат основой для выравнивания dashcam-потоков по аудиосигналу.

4.5 Формирование результата детекции

На данном этапе детекция возвращает аннотированное видео с полигонными масками (instance segmentation) через `results.plot()`. Маски кодируют два класса: `water` (класс 0) и `obstacle` (класс 1). Формирование bounding boxes, передача координат в навигационный модуль и построение карты препятствий остаются нереализованными.

---

5. Анализ развития ветки

История коммитов ветки `main`

Ветка `main` содержит всего три коммита:

Дата	SHA (prefix)	Сообщение	Тип
27.11.2025	`343a9d6`	`chore: initial commit and project structure setup`	Инициализация
01.12.2025	`1f865bd`	`build: migrate to uv and restructure source code layout`	Рефакторинг окружения
01.12.2025	`db11726`	`Add MIT License to the project`	Документация

Первый коммит создал структуру директорий и базовые конфигурации. Второй — мигрировал систему управления зависимостями с pip/venv на **uv** (ультрабыстрый менеджер пакетов от Astral SH) и реструктурировал layout исходного кода под `src/`-layout. Третий добавил лицензию через веб-интерфейс GitHub.

Текущее состояние ветки main

Реализовано:

- Структура директорий проекта согласно Cookiecutter Data Science
- Система управления зависимостями: uv + pyproject.toml + uv.lock
- Конфигурация pre-commit хуков: trailing whitespace, YAML-валидация, conventional commits
- Заготовка под конфигурацию Hydra (configs/config.yaml)
- Пустая точка входа main.py со стартовой заглушкой
- Метаданные академического проекта (CITATION.cff)

Обозначено как TODO или заглушка:

- main.py: весь функциональный код — заглушка print("Hello from diploma-project!")
- configs/config.yaml: абсолютно пустой файл
- setup.py: пустой файл (рудиментарный)
- src/usv/__init__.py: пустой модуль без экспортов
- models/: только .gitkeep, веса модели не добавлены
- Директории data/, notebooks/, reports/ — упомянуты в README, но физически отсутствуют

Технический долг

1. **Десинхронизация зависимостей:** библиотеки ultralytics, librosa, noisereduce, ffmpeg-python, tqdm используются в рабочих ветках, но не прописаны в pyproject.toml. При клонировании main и установке зависимостей через uv sync код из feat/-веток не запустится без ручного pip install.
2. **Отсутствует __init__.py в подпакетах:** пакеты data_loader, models, preprocessing в feat/-ветках не имеют инициализационных файлов — модули импортируются как скрипты, а не как пакеты, что ограничивает переиспользуемость кода.
3. **Ручные пути в скриптах:** большинство путей к файлам захардкожены в блоках if __name__ == "__main__". Для боевого применения их следует вынести в config.yaml через Hydra.

4. **Отсутствует CI/CD:** несмотря на наличие pre-commit конфигурации, в репозитории нет GitHub Actions для автоматического тестирования при пуше.
5. **device='cpu' в train_baseline.py**: флаг явно указывает на CPU-обучение как временное решение, что делает обучение на 50 эпохах крайне медленным. Необходима интеграция с CUDA или Google Colab.
6. **Разрыв ветки main и feat/-веток:** ветки feat/segmentation-dataset и feat/synchronization не объединены в main через Pull Request — весь функциональный код существует только в них. Это создаёт риск «потери» наработок при реорганизации репозитория.

Связь с другими ветками

Обе производные ветки разошлись от main после коммита db11726 (добавление лицензии, 01.12.2025) и развивались параллельно без слияний обратно в main.

feat/synchronization развила инфраструктурную часть из main: добавила data_loader/, preprocessing/ и visualization/. Основная работа — построение pipeline синхронизации трёх видеопотоков через аудио-корреляцию:

Коммит	Содержание
01.12.2025	AudioAligner с корректной знаковой конвенцией для сдвига
01.12.2025	Манифест-генератор с логикой временных меток телефона
01.12.2025–02.12.2025	Серия фиксов полосового фильтра (300→2500 Гц) и знака сдвига
02.12.2025	Финализация гибридного sync pipeline
02.12.2025	Удаление ручной коррекции аппаратной задержки
05.12.2025	Масштабирование синхронизации на весь датасет (FullDatasetResync)

feat/segmentation-dataset развила восприятие: добавила весь pipeline подготовки датасета и baseline-модель YOLOv8:

Коммит	Содержание
09.12.2025	Извлечение кадров с фиксом ротации телефона
10.12.2025	Baseline YOLOv8 pipeline (обучение + инференс)
10.12.2025	Super-resolution pipeline (EDSR/FSRCNN)
10.12.2025	Маскирование частей корпуса судна (ego-vehicle masking)
14.12.2025	Финальный тест извлечения кадров для разметки

Ветви развивают две ортогональные задачи: feat/synchronization — подготовку согласованных мультикамерных данных для стереозрения (preprocessing), feat/segmentation-dataset — восприятие отдельной камерой через обученную нейросеть (perception). Их интеграция является ключевой задачей для последующего развития в направлении стерео-детекции с пространственной локализацией.

Потенциальные точки роста

1. **Интеграция синхронизации и сегментации:** объединить SyncPipeline и YOLOv8 в единый preprocessing → perception конвейер, обрабатывающий синхронизированные стереопары.
2. **Модуль mapping:** реализация стереоматчинга (SGBM или нейросетевые методы типа RAFT-Stereo) для получения depth map и последующей 3D-локализации детектированных препятствий.
3. **Дообучение на собственном датасете:** текущий baseline обучен на MaSTr1325 — открытом датасете с USV-видео. Записанный собственный материал (центральная камера +

dashcams) при разметке может дать лучшую адаптацию к конкретным условиям эксплуатации судна.

4. **Конфигурируемость:** вынести все параметры (пути, пороги, размеры модели, частота кадров) в configs/config.yaml и подключить Hydra в main.py.
5. **Тесты:** файл pytest>=9.0.1 заявлен как dev-зависимость, но ни одного теста не написано — добавление unit-тестов для критических функций (find_offset, get_yolo_polygon, convert_mastr_to_yolo) снизит регрессионные риски.